UNIVERSIDAD DIEGO PORTALES



FACULTAD DE INGENIERÍA Y CIENCIAS

Tarea Nº2: Protocolo AMQP

Implementación con RabbitMQ y análisis de tráfico

Estudiantes:

Brandon Caravante

Nicolás Hermosillas

Grupo 16

Taller de Redes y Servicios 2026-2

Noviembre de 2026

Índice

1. Introducción **2**

2. Capítulo I: Protocolo e implementación **2**

2.1. Descripción y objetivo de AMQP 2
2.2. Conexiones, canales y confirmaciones 2
2.3. Tráfico esperado y puertos 3
2.4. Herramientas y topología 3
2.5. Instalación y configuración del servidor 4
2.6. Instalación y configuración del cliente 6
2.7. Procedimiento de ejecución 6

3. Capítulo II: Análisis de tráfico **6**

3.1. Metodología y procedencia de las evidencias 6
3.2. Negociación y apertura de sesión 7
3.3. P01: publicación y obtención de tres mensajes 8
3.4. P02: rechazo con reencolado y reentrega 9
3.5. P03: respuesta ante una cola inexistente 11
3.6. Comparación entre tráfico esperado y observado 11
3.7. Hipótesis sobre modificaciones del tráfico 12
3.8. Alcance y limitaciones 13

4. Conclusiones **14**

Referencias **15**

A. Anexos **16**

A.1. Ampliaciones de campos relevantes 16
A.2. Configuración reproducible 18
A.3. Archivos de respaldo y reproducción 20

1. Introducción

La experiencia consistió en implementar y estudiar una comunicación mediante AMQP 0-9-1 entre un servidor RabbitMQ y una aplicación cliente escrita en Python con la biblioteca Pika. Ambos programas se ejecutaron en contenedores Linux administrados por Docker Desktop desde Windows. El objetivo fue comprobar el funcionamiento del servicio y relacionar las acciones de la aplicación con los mensajes intercambiados en la red.

Se realizaron tres escenarios: publicación y obtención de mensajes, rechazo con reencolado y consulta de una cola inexistente. La captura se obtuvo con tcpdump dentro del contenedor cliente y se analizó con tshark, herramienta de línea de comandos de Wireshark, y con la interfaz gráfica de Wireshark. Este procedimiento permitió conservar un archivo PCAP independiente del tiempo de vida del cliente.

El primer capítulo presenta el protocolo, el entorno y la instalación. El segundo describe la metodología y analiza las pruebas. Finalmente, se formulan hipótesis sobre modificaciones del tráfico y se exponen las conclusiones. Las evidencias gráficas y las tablas acompañan las explicaciones correspondientes. Al final se incluye un anexo con ampliaciones de los campos relevantes y los archivos de configuración para facilitar su lectura y reproducción.

2. Capítulo I: Protocolo e implementación

2.1. Descripción y objetivo de AMQP

AMQP 0-9-1 permite intercambiar mensajes entre aplicaciones y un intermediario o *broker*. Los productores publican en un *exchange*, que encamina los mensajes hacia colas según reglas de asociación. Los receptores pueden suscribirse o solicitar mensajes bajo demanda. Esta separación permite desacoplar el envío de su procesamiento [1].

En el laboratorio se usó el exchange predeterminado, identificado por un nombre vacío. Cada cola queda asociada a él mediante una clave de enrutamiento igual a su nombre. Por tanto, el cliente publica utilizando como *routing key* el nombre que devolvió RabbitMQ al crear la cola [1].

El cliente desarrollado cumple tanto el rol de productor como el de receptor. Obtiene mensajes mediante `Basic.Get`; no usa una suscripción `Basic.Consume`. RabbitMQ actúa como un programa separado que recibe, administra y entrega esos mensajes. La selección de Pika se realizó tras la aclaración recibida por el grupo respecto de que el énfasis de la actividad es trabajar el protocolo; el tutorial oficial de RabbitMQ también emplea esta biblioteca [2].

2.2. Conexiones, canales y confirmaciones

Un canal AMQP es una comunicación lógica dentro de una conexión TCP. Las operaciones de cola y mensajería se realizan en canales. Esta distinción importa porque un error puede cerrar un canal sin terminar inmediatamente toda la conexión [3].

Se habilitaron confirmaciones de publicación y confirmaciones manuales de recepción. Las primeras viajan del broker al productor; las segundas, del receptor al broker. Son mecanismos independientes y no deben confundirse con los acuses de recibo TCP [4]. La dirección del paquete permite interpretar el papel de cada `Basic.Ack` observado.

Para el segundo escenario se utilizó `Basic.Nack` con `requeue=True`, solicitando devolver el mensaje a la cola. Este método es una extensión de RabbitMQ a AMQP 0-9-1 [4].

2.3. Tráfico esperado y puertos

Se esperaba observar el establecimiento TCP, la negociación AMQP, la apertura de conexión y canal, las operaciones de cola, la transferencia de contenido y el cierre de sesión. El servicio AMQP del laboratorio utiliza TCP 5672 sin TLS. El panel web de administración usa 15672 y tiene un propósito distinto: revisar el estado del broker [5].

Las colas de las pruebas son exclusivas, tienen nombres generados por el servidor y pertenecen a la conexión que las declara. Se eliminan al cerrarse esa conexión [6]. Por esta razón, el panel puede no mostrar colas después de terminar las pruebas, aunque el intercambio haya sido correcto.

2.4. Herramientas y topología

El host fue Windows con Docker Desktop y backend Linux. Se registraron Docker Engine 29.7.2 y Docker Compose v5.5.1. El servidor utilizó la imagen `rabbitmq:4.4.0-management`; el cliente ejecutó Python 3.12.16, Pika 1.3.2 y tcpdump 4.99.3. El análisis automatizado se realizó con tshark 4.6.8 y se complementó con inspección gráfica en Wireshark.

Compose creó la red `taller2-amqp_default`. En la captura analizada, RabbitMQ tuvo la dirección 172.18.0.2 y el cliente 172.18.0.3. Son direcciones de esa ejecución, no valores fijos del proyecto. La Figura 1 representa el entorno; la Tabla 1 resume los componentes.

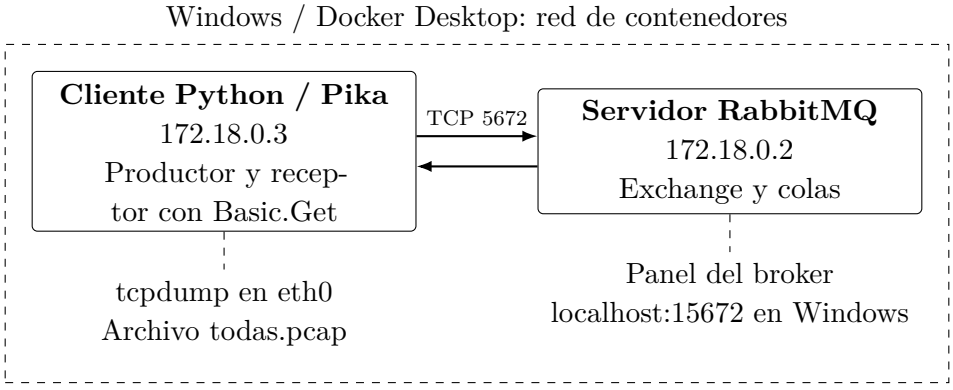


Figura 1: Topología de la ejecución analizada. Las direcciones pueden cambiar al recrear la red. Elaboración propia a partir de Compose y del PCAP.

Tabla 1: Componentes y parámetros del entorno registrado.

Elemento	Valor observado o configurado
Servidor	RabbitMQ 4.4.0, variante management
Cliente	Python 3.12.16 y Pika 1.3.2
Contenedores	Docker Engine 29.7.2; Compose v5.5.1
Captura y análisis	tcpdump 4.99.3; tshark 4.6.8
Protocolo	AMQP 0-9-1, TCP 5672, sin TLS
Administración	HTTP 15672, publicado en loopback del host
Conexión	Host rabbitmq; usuario taller; vhost /
Evidencia principal	todas.pcap, 121 paquetes

2.5. Instalación y configuración del servidor

El archivo `compose.yaml` define el servicio `rabbitmq`, su imagen y las credenciales de demostración. El nombre del servicio permite al cliente conectarse usando `rabbitmq` como host. Se utiliza el vhost predeterminado `/`. La Figura 2 muestra la configuración de ambos servicios en el editor.

D: > taller > tarea2 > compose.yaml

```yaml
name: taller2-amqp
services:
  rabbitmq:
    image: rabbitmq:4.4.0-management
    hostname: rabbitmq
    environment:
      RABBITMQ_DEFAULT_USER: taller
      RABBITMQ_DEFAULT_PASS: taller-local-2026
    ports:
      - "127.0.0.1:15672:15672"
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 5s
      timeout: 10s
      retries: 24
      start_period: 20s
  cliente:
    build: ./cliente
    environment:
      AMQP_HOST: rabbitmq
      AMQP_USER: taller
      AMQP_PASSWORD: taller-local-2026
    depends_on:
      rabbitmq:
        condition: service_healthy
    volumes:
      - ./evidencias:/evidencias
    cap_add: [NET_RAW]
    profiles: [pruebas]

```

Figura 2: Archivo Compose en el editor. Se observan ambos servicios, sus variables, el control de salud y el montaje de evidencias.

El servidor tiene un control de salud basado en `rabbitmq-diagnostics-qping`. La

condición `service_healthy` impide iniciar las pruebas antes de que el servicio esté preparado. La Figura 3 comprueba el arranque y el estado saludable.

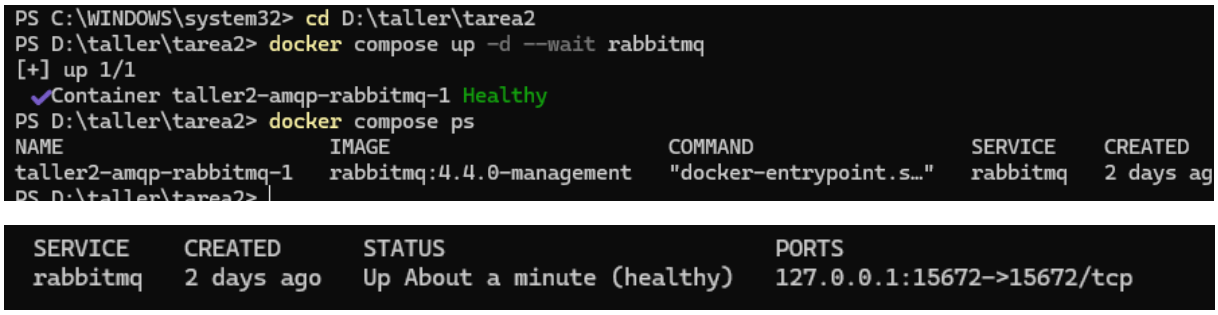


Figura 3: RabbitMQ iniciado y saludable. Dos ampliaciones complementarias de la misma captura permiten leer el comando, el contenedor y la columna de estado.

El puerto 5672 se mantiene en la red de contenedores; únicamente se publica el panel en `127.0.0.1:15672`, accesible desde el equipo local. Las credenciales declaradas son exclusivas de demostración. El listado completo está en el Anexo A.2 y el digest de la imagen en el Anexo A.3.

La Figura 4 muestra el panel del broker el 8 de noviembre. Sus contadores vacíos corresponden al estado de reposo después de usar clientes y colas temporales; esta vista verifica el servicio, pero no sustituye el análisis de paquetes.

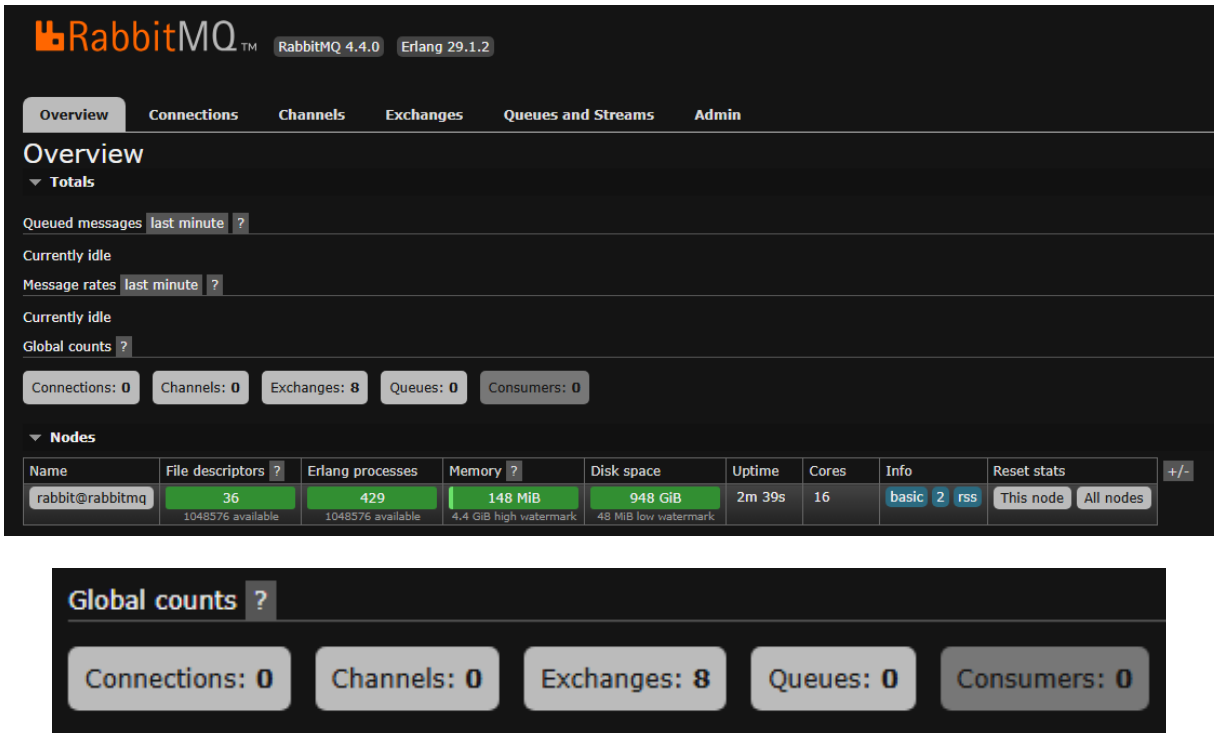


Figura 4: Panel Overview de RabbitMQ 4.4.0, con una ampliación de los contadores. La ausencia de conexiones, colas y consumidores es compatible con la finalización de las pruebas temporales.

2.6. Instalación y configuración del cliente

La imagen cliente se construye con el Dockerfile incluido. Parte de `python:3.12-slim-bookworm`, instala tcpdump y fija la dependencia `pika==1.3.2`. El programa recibe host, usuario y contraseña mediante variables de entorno.

El directorio `evidencias` del host se monta en `/evidencias`. Así, los resultados permanecen disponibles cuando se elimina el contenedor temporal. La captura requiere acceso a paquetes de red; la configuración incluye `NET_RAW`. Los archivos de construcción y captura se presentan en el Anexo A.2.

2.7. Procedimiento de ejecución

La ejecución completa se realizó desde PowerShell, en la carpeta del proyecto:

```
cd D:\taller\tarea2
& .\scripts\ejecutar.ps1
```

El script comprueba el motor Docker, inicia RabbitMQ, construye el cliente y ejecuta las pruebas. Sus operaciones centrales son:

```
docker compose up -d --wait rabbitmq
docker compose build cliente
docker compose run --rm cliente todas
```

Al finalizar se conservan la captura, los resultados JSON y los registros. RabbitMQ permanece activo; el cliente sale y su contenedor se elimina mediante `--rm`. Para repetir una sola prueba puede sustituirse `todas` por `P01`, `P02` o `P03`.

Durante la preparación se resolvió un fallo de inicio de Docker causado por sockets temporales inaccesibles, conservando sus directorios como respaldo y permitiendo su regeneración. Posteriormente el servidor alcanzó el estado saludable y las tres pruebas pudieron ejecutarse. No se necesitó instalar Linux como sistema operativo principal.

3. Capítulo II: Análisis de tráfico

3.1. Metodología y procedencia de las evidencias

La ejecución analizada se realizó el 6 de noviembre de 2026 a las 22:06:49, hora de Santiago. Los registros JSON expresan la hora en UTC, equivalente al 7 de noviembre a las 01:06:49 UTC. No se trata de una segunda ejecución.

El 8 de noviembre se repitieron individualmente las tres pruebas. Los JSON registran las 22:27:45, 22:28:03 y 22:28:19 de Santiago, respectivamente (9 de noviembre, 01:27:45–01:28:19 UTC). Se conservan como evidencia adicional en `P01.pcap`, `P02.pcap`, `P03.pcap` y sus resultados JSON. Las capturas de terminal de las Figuras 6 y 8 pertenecen a esta repetición, mientras las capturas gráficas de Wireshark abren el archivo original `todas.pcap`.

Se inició tcpdump en la interfaz `eth0` del cliente antes de abrir las conexiones, con el filtro de captura `tcpport5672`, paquetes completos y escritura inmediata al archivo. Se detuvo con SIGINT después de las pruebas. La ubicación dentro del contenedor evitó depender de una interfaz virtual visible en Windows.

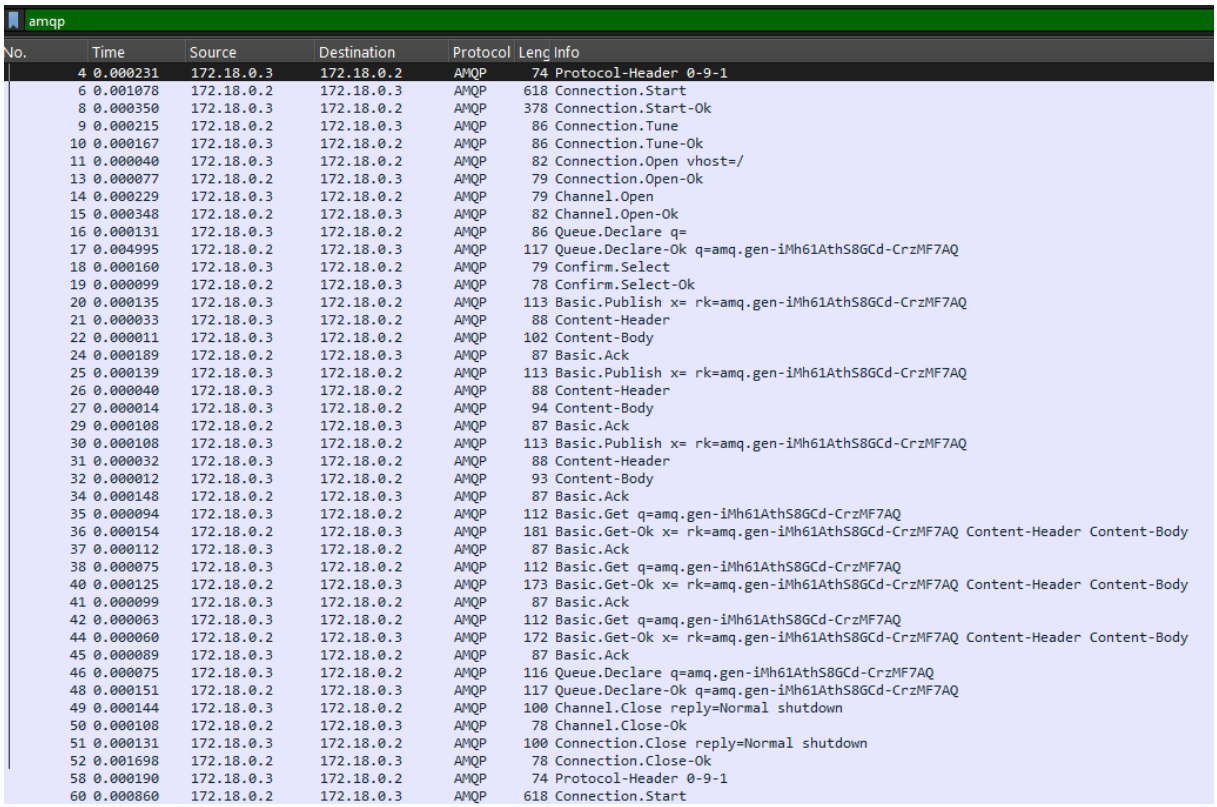
El archivo `todas.pcap` contiene 121 paquetes. tcpdump informó 121 recibidos por el filtro y cero descartados por el kernel. Esta cifra describe al capturador local; no demuestra ausencia general de pérdida o fallas en cualquier red. El filtro de visualización `amqp` seleccionó 84 tramas en tshark 4.6.8. En la captura gráfica aportada, Wireshark muestra 87, incluyendo los tres encabezados de protocolo de las sesiones (tramas 4, 58 y 98). Se trata de una diferencia de decodificación y no de un cambio del total de 121 paquetes. Los resultados se resumen en la Tabla 3 y la Figura 13.

La Figura 5 muestra la inspección gráfica general. Todos los números de trama de este informe se refieren a esa captura original. Si se repite el experimento, pueden cambiar la segmentación TCP, las direcciones y la numeración.

3.2. Negociación y apertura de sesión

En P01, las primeras tres tramas muestran el establecimiento TCP. A continuación se intercambian los mensajes de negociación AMQP: `Connection.Start` en la trama 6, `Start-Ok` en la 8, `Tune` en la 9 y `Tune-Ok` en la 10. La solicitud `Connection.Open` aparece en la 11 y su respuesta en la 13.

Las tramas 14 y 15 corresponden a `Channel.Open` y `Channel.Open-Ok`. Después se declara la cola, mediante las tramas 16 y 17, y se habilitan confirmaciones con `Confirm.Select` y su respuesta en 18 y 19. La secuencia sitúa las operaciones de mensajería después de preparar conexión, canal y cola.



amqp

No.	Time	Source	Destination	Protocol	Leng	Info
4	0.000231	172.18.0.3	172.18.0.2	AMQP	74	Protocol-Header 0-9-1
6	0.001078	172.18.0.2	172.18.0.3	AMQP	618	Connection.Start
8	0.000350	172.18.0.3	172.18.0.2	AMQP	378	Connection.Start-Ok
9	0.000215	172.18.0.2	172.18.0.3	AMQP	86	Connection.Tune
10	0.000167	172.18.0.3	172.18.0.2	AMQP	86	Connection.Tune-Ok
11	0.000040	172.18.0.3	172.18.0.2	AMQP	82	Connection.Open vhost=/
13	0.000077	172.18.0.2	172.18.0.3	AMQP	79	Connection.Open-Ok
14	0.000229	172.18.0.3	172.18.0.2	AMQP	79	Channel.Open
15	0.000348	172.18.0.2	172.18.0.3	AMQP	82	Channel.Open-Ok
16	0.000131	172.18.0.3	172.18.0.2	AMQP	86	Queue.Declare q=
17	0.004995	172.18.0.2	172.18.0.3	AMQP	117	Queue.Declare-Ok q=amq.gen-iMh61AthS8GCd-CrzMF7AQ
18	0.000160	172.18.0.3	172.18.0.2	AMQP	79	Confirm.Select
19	0.000099	172.18.0.2	172.18.0.3	AMQP	78	Confirm.Select-Ok
20	0.000135	172.18.0.3	172.18.0.2	AMQP	113	Basic.Publish x= rk=amq.gen-iMh61AthS8GCd-CrzMF7AQ
21	0.000033	172.18.0.3	172.18.0.2	AMQP	88	Content-Header
22	0.000011	172.18.0.3	172.18.0.2	AMQP	102	Content-Body
24	0.000189	172.18.0.2	172.18.0.3	AMQP	87	Basic.Ack
25	0.000139	172.18.0.3	172.18.0.2	AMQP	113	Basic.Publish x= rk=amq.gen-iMh61AthS8GCd-CrzMF7AQ
26	0.000040	172.18.0.3	172.18.0.2	AMQP	88	Content-Header
27	0.000014	172.18.0.3	172.18.0.2	AMQP	94	Content-Body
29	0.000108	172.18.0.2	172.18.0.3	AMQP	87	Basic.Ack
30	0.000108	172.18.0.3	172.18.0.2	AMQP	113	Basic.Publish x= rk=amq.gen-iMh61AthS8GCd-CrzMF7AQ
31	0.000032	172.18.0.3	172.18.0.2	AMQP	88	Content-Header
32	0.000012	172.18.0.3	172.18.0.2	AMQP	93	Content-Body
34	0.000148	172.18.0.2	172.18.0.3	AMQP	87	Basic.Ack
35	0.000094	172.18.0.3	172.18.0.2	AMQP	112	Basic.Get q=amq.gen-iMh61AthS8GCd-CrzMF7AQ
36	0.000154	172.18.0.2	172.18.0.3	AMQP	181	Basic.Get-Ok x= rk=amq.gen-iMh61AthS8GCd-CrzMF7AQ Content-Header Content-Body
37	0.000112	172.18.0.3	172.18.0.2	AMQP	87	Basic.Ack
38	0.000075	172.18.0.3	172.18.0.2	AMQP	112	Basic.Get q=amq.gen-iMh61AthS8GCd-CrzMF7AQ
40	0.000125	172.18.0.2	172.18.0.3	AMQP	173	Basic.Get-Ok x= rk=amq.gen-iMh61AthS8GCd-CrzMF7AQ Content-Header Content-Body
41	0.000099	172.18.0.3	172.18.0.2	AMQP	87	Basic.Ack
42	0.000063	172.18.0.3	172.18.0.2	AMQP	112	Basic.Get q=amq.gen-iMh61AthS8GCd-CrzMF7AQ
44	0.000060	172.18.0.2	172.18.0.3	AMQP	172	Basic.Get-Ok x= rk=amq.gen-iMh61AthS8GCd-CrzMF7AQ Content-Header Content-Body
45	0.000089	172.18.0.3	172.18.0.2	AMQP	87	Basic.Ack
46	0.000075	172.18.0.3	172.18.0.2	AMQP	116	Queue.Declare q=amq.gen-iMh61AthS8GCd-CrzMF7AQ
48	0.000151	172.18.0.2	172.18.0.3	AMQP	117	Queue.Declare-Ok q=amq.gen-iMh61AthS8GCd-CrzMF7AQ
49	0.000144	172.18.0.3	172.18.0.2	AMQP	100	Channel.Close reply=Normal shutdown
50	0.000108	172.18.0.2	172.18.0.3	AMQP	78	Channel.Close-Ok
51	0.000131	172.18.0.3	172.18.0.2	AMQP	100	Connection.Close reply=Normal shutdown
52	0.001698	172.18.0.2	172.18.0.3	AMQP	78	Connection.Close-Ok
58	0.000190	172.18.0.3	172.18.0.2	AMQP	74	Protocol-Header 0-9-1
60	0.000860	172.18.0.2	172.18.0.3	AMQP	618	Connection.Start

Figura 5: Vista general de todas.pcap con filtro AMQP. Se reconocen el encabezado del protocolo, la negociación, la apertura de canal, la declaración de cola y las operaciones de mensajes.

3.3. P01: publicación y obtención de tres mensajes

El cliente publicó los textos “Hola desde Brandon y Nicolas”, “Segundo mensaje AMQP” y “Tercer mensaje AMQP”. Posteriormente los obtuvo con confirmación manual, verificó contenido y orden, y comprobó un contador de mensajes disponibles igual a cero.

```
PS C:\Users\Brandon> cd D:\taller\tarea2
PS D:\taller\tarea2> docker compose run --rm cliente P01
[+] run 1/1
 ✔Container taller2-amqp-rabbitmq-1 Running
Container taller2-amqp-rabbitmq-1 Waiting
Container taller2-amqp-rabbitmq-1 Healthy
Container taller2-amqp-cliente-run-016f9c024f49 Creating
Container taller2-amqp-cliente-run-016f9c024f49 Created
```

Figura 6: Ejecución individual de P01 el 8 de noviembre. Se amplía la zona de comandos y arranque; el resultado íntegro está en el JSON adjunto.

La repetición del 8 de noviembre registró resultado OK, tres mensajes recibidos y contador final cero. La captura original se analiza a continuación.

Las publicaciones aparecen en las tramas 20, 25 y 30. Las confirmaciones del broker se observan en 24, 29 y 34. Las respuestas `Basic.Get-Ok` que transportan los mensajes se localizan en 36, 40 y 44; las confirmaciones del receptor, en 37, 41 y 45. La Figura 7 muestra un extracto de la decodificación y la Tabla 2 identifica los eventos.

El resultado JSON contiene las listas de mensajes publicados y recibidos, que coinciden exactamente en esta prueba. La aplicación terminó con `resultado:OK`. Esta observación corresponde a tres mensajes en una cola y un cliente; no establece garantías de orden para configuraciones diferentes.

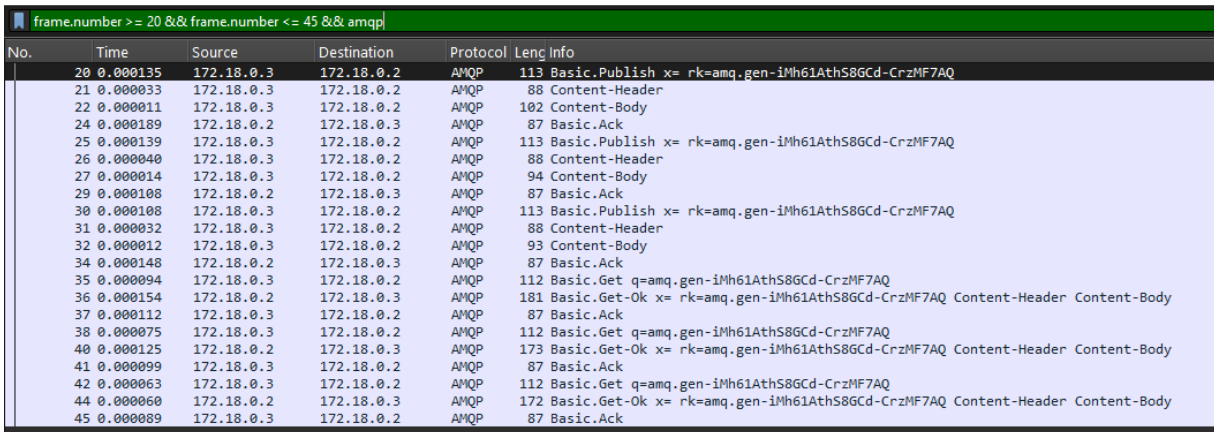


Figura 7: P01: filtro sobre las tramas 20 a 45 de todas.pcap. Se observan las publicaciones, sus confirmaciones y las solicitudes y respuestas de obtención de mensajes.

Tabla 2: Eventos representativos de todas.pcap.

Prueba	Tramas	Interpretación
P01	6–15	Negociación y apertura de canal
P01	20, 25, 30	Publicaciones del cliente
P01	24, 29, 34	Confirmaciones del broker al productor
P01	36, 40, 44	Obtención de mensajes con contenido
P01	37, 41, 45	Confirmaciones del receptor al broker
P02	81, 84, 85	Rechazo, reentrega y confirmación final
P03	114–116	Consulta pasiva y cierre de canal con 404
P03	117, 119	Cierre normal posterior de la conexión

3.4. P02: rechazo con reencolado y reentrega

Se publicó “Mensaje para reentrega” y se obtuvo sin confirmación automática. La primera entrega registró `redelivered=False`. El cliente solicitó el reencolado con `Basic.Nack` y volvió a obtener el mismo cuerpo; la segunda entrega registró `redelivered=True`. Finalmente confirmó la entrega y verificó la cola vacía.

```
PS C:\Users\Brandon> cd D:\taller\tarea2
PS D:\taller\tarea2> docker compose run --rm cliente P02
[+] run 1/1
 ✔Container taller2-amqp-rabbitmq-1 Running
Container taller2-amqp-rabbitmq-1 Waiting
Container taller2-amqp-rabbitmq-1 Healthy
Container taller2-amqp-cliente-run-b23a48e3f1c4 Creating
Container taller2-amqp-cliente-run-b23a48e3f1c4 Created
```

Figura 8: Ejecución individual de P02 en la misma sesión. La salida completa se conserva en la captura original y en el JSON.

La trama 81 contiene el rechazo con `Requeue:True`; la 84 contiene `Basic.Get-Ok` con `Redelivered:True`. La trama 85 confirma esa última entrega. Los campos se verificaron con tshark y en la interfaz gráfica: la Figura 9 muestra la lista de ambos eventos; las Figuras 10 y 11 amplían los indicadores de reencolado y reentrega.

La reentrega se provocó mediante una operación válida del cliente. No se modificaron paquetes en tránsito ni se simularon pérdidas de red. La prueba demuestra la relación entre el rechazo explícito, el retorno a la cola y la nueva obtención del mensaje.

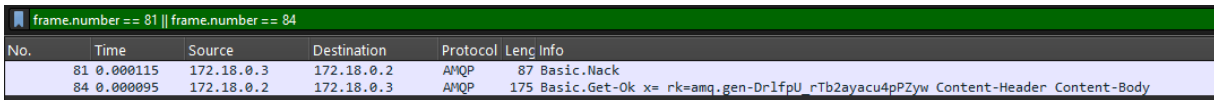


Figura 9: P02: las tramas 81 y 84 muestran el rechazo del cliente y la entrega posterior del servidor.

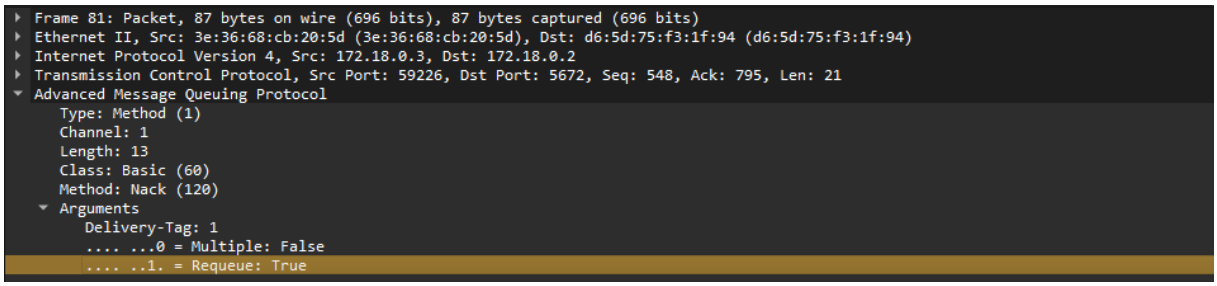


Figura 10: Detalle de la trama 81. El método Basic.Nack incluye el indicador Requeue verdadero, que solicita devolver el mensaje a la cola.

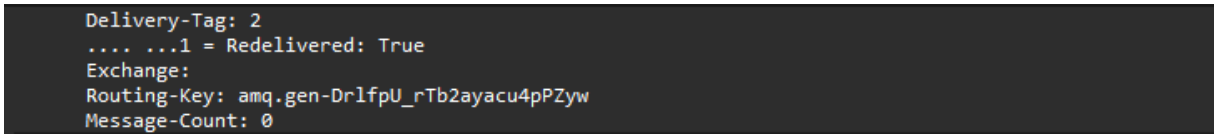


Figura 11: Detalle de la trama 84, con Delivery-Tag 2 y Redelivered verdadero. El mensaje fue entregado nuevamente después del rechazo.

El Anexo A.1 presenta estos dos indicadores a mayor escala para su inspección.

3.5. P03: respuesta ante una cola inexistente

Para generar un error controlado, el cliente declaró una cola temporal, la eliminó y consultó ese nombre mediante declaración pasiva. La creación se observa en 110–111 y la eliminación en 112–113. La nueva consulta de la trama 114 produce `Channel.Close` en la 115, con `Reply-Code:404` y texto `NOT_FOUND`.

El cliente responde `Channel.Close-Ok` en la trama 116. La aplicación verifica que la conexión sigue abierta y luego la termina normalmente: `Connection.Close` en 117 y `Connection.Close-Ok` en 119. La Figura 12 documenta el error y el cierre posterior. Se distingue así el alcance del error de canal respecto del ciclo de vida de la conexión.

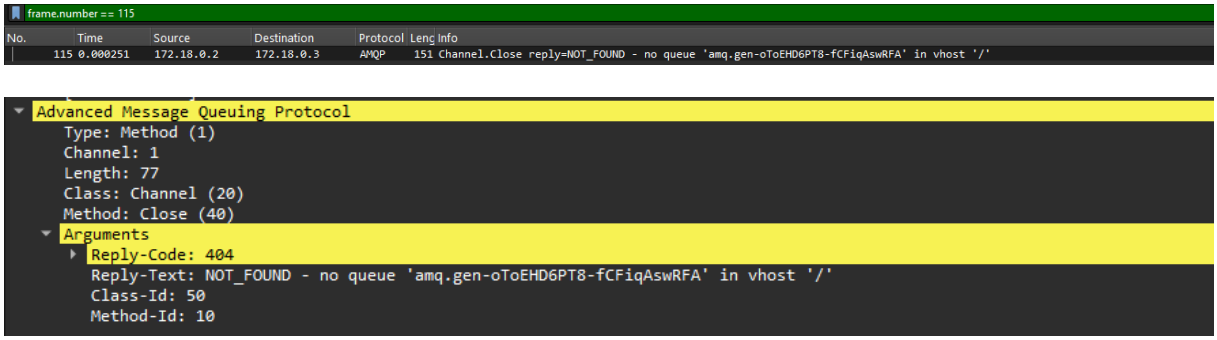


Figura 12: P03: vista de la trama 115 y ampliación de los argumentos AMQP. RabbitMQ devuelve Channel.Close con Reply-Code 404 y el texto NOT_FOUND por una cola inexistente.

El código de respuesta se amplía también en el Anexo A.1.

3.6. Comparación entre tráfico esperado y observado

Las tres pruebas terminaron correctamente según sus criterios, incluido el error esperado en P03. Se observaron negociación, apertura de canal, operaciones de cola, transferencia de mensajes, confirmaciones y cierres. El análisis de red respalda las comprobaciones de la aplicación; la Tabla 4 reúne esa correspondencia.

Una trama TCP puede transportar más de una unidad AMQP. Por ejemplo, la trama 36 contiene `Basic.Get-Ok`, cabecera y cuerpo. Por tanto, no se interpreta el total de paquetes como número de mensajes publicados. Tampoco se confunden los acuses TCP con los métodos de confirmación AMQP.

El laboratorio no utiliza TLS, de modo que el archivo contiene mensajes y autenticación de demostración en claro. El tráfico HTTP del panel de administración queda fuera del filtro de captura. La ausencia de colas en el panel tras finalizar es compatible con el uso de colas exclusivas y clientes temporales.

Resultados reales de la aplicacion cliente

Fuente: todas-resultados.json | Resumen de los campos verificados

```
P01 | resultado: OK
  Mensajes publicados: 3 | recibidos: 3 | cola final: 0
P02 | resultado: OK
  Primera redelivered: false | segunda redelivered: true
P03 | resultado: OK
  Codigo: 404 | conexion_sigue_abierta: true

Ejecucion: 06-11-2026, 22:06:49 (America/Santiago)
```

Figura 13: Resumen generado a partir del JSON de resultados reales. No corresponde a una captura de interfaz gráfica.

Tabla 3: Criterios verificados por la aplicación.

Prueba	Criterio	Resultado observado
P01	Recepción exacta y cola disponible vacía	Tres mensajes coincidentes, contador 0; OK
P02	Reencolado y posterior confirmación	Segunda entrega con redelivered verdadero; OK
P03	Cola inexistente y alcance del error	Código 404; conexión todavía abierta; OK

Tabla 4: Comparación entre comportamiento esperado y evidencia.

Etapa	Tráfico esperado	Evidencia observada
Inicio	Negociación, apertura y declaración	Start, Tune, Open, Channel.Open, Queue.Declare
P01	Publicación y obtención confirmadas	Publish, Get-Ok y Ack en ambos sentidos
P02	Rechazo y reentrega	Nack: requeue verdadero; Get-Ok: redelivered verdadero
P03	Error por consulta pasiva sin cola	Channel.Close: 404 y Close-Ok
Final	Cierre de conexión	Connection.Close y Close-Ok

3.7. Hipótesis sobre modificaciones del tráfico

Las siguientes propuestas no fueron ejecutadas y se plantean como base para el capítulo posterior de modificación de tráfico.

Cambio de clave de enrutamiento. Si una publicación se redirige a una clave sin cola asociada, se espera que el destino efectivo cambie o que no exista ruta. Con el indicador obligatorio habilitado, se buscaría una devolución al productor. Se comprobarían el método de respuesta y el resultado de la aplicación, verificando previamente la configuración del exchange [1].

Retardo o pérdida de paquetes. Introducir retardo o descartar segmentos podría aumentar el tiempo de las operaciones y ocasionar retransmisiones o fallas al superar los límites configurados. La comprobación debería separar eventos TCP, tiempos de aplicación y timeouts. Esta ejecución no proporciona métricas para cuantificar esos efectos.

Pérdida de una confirmación de recepción. En una prueba futura con una cola no exclusiva que permanezca tras la desconexión, una entrega no confirmada podría volver a estar disponible al perderse la conexión. Se observarían su estado y la posterior entrega [4]. Esta hipótesis requiere cambiar la configuración: las colas exclusivas actuales se eliminan cuando termina su conexión y no permiten evaluar ese escenario directamente [6].

3.8. Alcance y limitaciones

Se evaluó un único cliente que publica y obtiene bajo demanda, con colas temporales y pocos mensajes. No se midieron rendimiento sostenido, concurrencia, alta disponibilidad ni persistencia frente a reinicios. Las evidencias combinan capturas originales de terminal, editor, panel y Wireshark, junto con tablas y un resumen procesado del JSON. Los recortes del PDF amplían zonas relevantes; las once imágenes originales se conservan sin modificaciones en el proyecto. El PCAP original se incluye para permitir su revisión independiente.

4. Conclusiones

Se implementó una comunicación AMQP 0-9-1 con RabbitMQ y Python/Pika en contenedores separados. La ejecución real verificó el intercambio de tres mensajes, la reentrega después de un rechazo con reencolado y la respuesta de error ante una cola inexistente.

El análisis permitió relacionar cada acción con métodos y campos concretos. En particular, se diferenciaron las confirmaciones del broker y del receptor, el indicador de reentrega y el cierre de un canal frente al cierre de una conexión. La captura y los resultados JSON coinciden en los tres escenarios.

El procedimiento es reproducible mediante los archivos entregados y conserva sus evidencias fuera del contenedor temporal. Los resultados se limitan al entorno y a las operaciones descritas. Las modificaciones del tráfico, su cuantificación y las pruebas de rendimiento quedan como trabajo posterior; no se presentan como actividades ya realizadas.

Referencias

[1] RabbitMQ. *AMQP 0-9-1 Model Explained*. https://www.rabbitmq.com/tutorials/amqp-concepts.

[2] RabbitMQ. *Tutorial: Hello World, Python*. https://www.rabbitmq.com/tutorials/tutorial-one-python.

[3] RabbitMQ. *Channels*. https://www.rabbitmq.com/docs/channels.

[4] RabbitMQ. *Consumer Acknowledgements and Publisher Confirms*. https://www.rabbitmq.com/docs/confirms.

[5] RabbitMQ. *Networking and RabbitMQ*. https://www.rabbitmq.com/docs/networking.

[6] RabbitMQ. *Queues*. https://www.rabbitmq.com/docs/queues.

[7] Docker Hub. *RabbitMQ: imagen oficial*. https://hub.docker.com/_/rabbitmq.

[8] Taller de Redes y Servicios. *Enunciado Tarea 2* y *Template Informe Tareas Taller de Redes*. Materiales del curso proporcionados al grupo.

Documentación web revisada el 8 de noviembre de 2026.

A. Anexos

A.1. Ampliaciones de campos relevantes

Las imágenes del desarrollo permiten seguir las pruebas mientras se lee el análisis. Este anexo repite únicamente los detalles que requieren mayor ampliación. Se utiliza la resolución original de los archivos aportados; no se altera ni reconstruye su contenido.

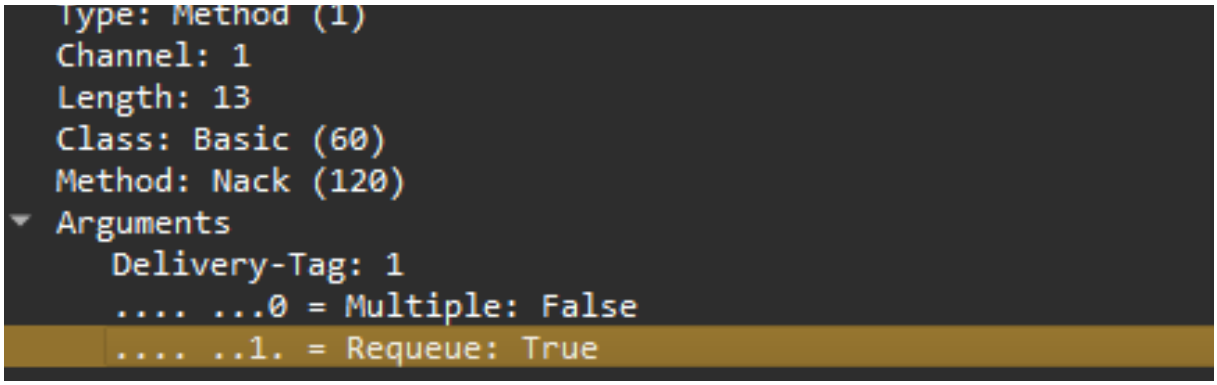


Figura 14: Ampliación de Basic.Nack en la trama 81: clase, método, identificador de entrega e indicador Requeue verdadero.

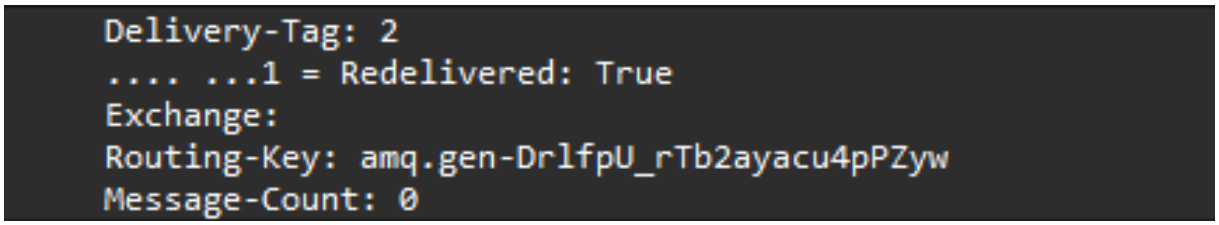


Figura 15: Ampliación de la trama 84: Delivery-Tag 2 y Redelivered verdadero, junto con el destino de enrutamiento.

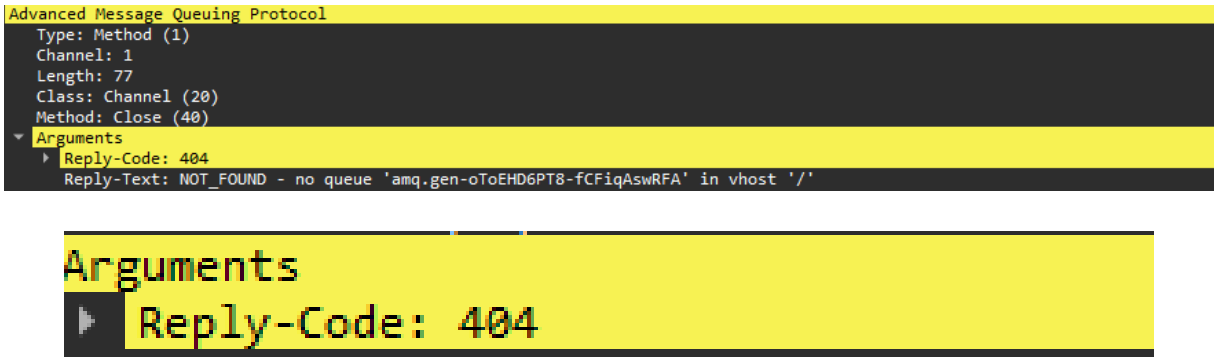


Figura 16: Ampliación del cierre de canal de la trama 115 y del código 404. El texto completo de NOT_FOUND permanece en la captura original y en la figura del desarrollo.

A.2. Configuración reproducible

Archivo Compose utilizado. Define el servidor y la construcción del cliente, las variables de conexión, la condición de salud y el montaje de evidencias. La contraseña corresponde únicamente a este laboratorio.

```
name: taller2-amqp
services:
  rabbitmq:
    image: rabbitmq:4.4.0-management
    hostname: rabbitmq
    environment:
      RABBITMQ_DEFAULT_USER: taller
      RABBITMQ_DEFAULT_PASS: taller-local-2026
    ports:
      - "127.0.0.1:15672:15672"
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 5s
      timeout: 10s
      retries: 24
      start_period: 20s
  cliente:
    build: ./cliente
    environment:
      AMQP_HOST: rabbitmq
      AMQP_USER: taller
      AMQP_PASSWORD: taller-local-2026
    depends_on:
      rabbitmq:
        condition: service_healthy
    volumes:
      - ./evidencias:/evidencias
    cap_add: [NET_RAW]
    profiles: [pruebas]
```

Dockerfile del cliente.

```
FROM python:3.12-slim-bookworm
RUN apt-get update && apt-get install -y --no-install-recommends tcpdump \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY laboratorio.py capturar.sh ./
ENTRYPOINT ["sh", "/app/capturar.sh"]
```

Dependencia fijada.

```
pika==1.3.2
```

Captura y ejecución dentro del cliente.

```
#!/bin/sh
set -eu
case "${1:-todas}" in todas|P01|P02|P03) prueba="${1:-todas}";; *) echo 'Use todas,
    P01, P02 o P03'; exit 2;; esac
tcpdump -i eth0 -p -Z root -U -s 0 -w "/evidencias/${prueba}.pcap" 'tcp port 5672' >"/
    evidencias/${prueba}-captura.log" 2>&1 &
captura=$!
finalizar() { kill -INT "$captura" 2>/dev/null || true; wait "$captura" || true; }
trap finalizar EXIT
sleep 1
kill -0 "$captura"
python -u /app/laboratorio.py "$prueba"
sleep 1
```

A.3. Archivos de respaldo y reproducción

Se incluyen además las once capturas originales, un manifiesto SHA-256 y las evidencias individuales de la repetición del 8 de noviembre. El proyecto contiene el programa completo `codigo/cliente/laboratorio.py` y el ejecutor `codigo/scripts/ejecutar.ps1`. Se entregan como archivos editables, junto con los listados anteriores.

La imagen del servidor usada en la ejecución fue:

```
rabbitmq@sha256:af6f50d38098d8bd52fef7f3fa85015dd8b2bf6def58e995bf58
4ae9ba0f6407
```

La etiqueta de la imagen base Python puede actualizarse en el registro. Las versiones registradas en este informe describen la ejecución analizada; reconstrucciones futuras deben volver a registrar sus versiones.

Tabla 5: Archivos de evidencia incluidos en el proyecto.

Archivo	Contenido
`todas.pcap`	Captura original de las tres pruebas
`todas-resultados.json`	Comprobaciones y resultados de aplicación
`todas-captura.log`	Estadísticas finales de tcpdump
`todas-cliente.log`	Salida de la ejecución cliente
`servidor.log`	Registros de RabbitMQ
`amqp-tramas.tsv`	Índice de tramas AMQP
`todas-tramas.tsv`	Índice completo, incluido TCP
`trama-81.txt`	Decodificación del rechazo con reencolado
`trama-84.txt`	Decodificación de la reentrega
`trama-115.txt`	Decodificación del error de canal 404

Para abrir la evidencia en Wireshark se selecciona `todas.pcap` y se aplica el filtro `amqp`. El filtro `tcp.port==5672` incluye también establecimiento y cierre TCP. Las referencias de trama se contrastan contra este archivo y no deben trasladarse sin revisión a capturas posteriores.

Los archivos incluidos permiten reproducir las pruebas y contrastar el análisis de paquetes con los resultados de la aplicación. El video de demostración y el repositorio del grupo complementan el presente documento.